Flood Polynomial Model — Test Results

Automated Test Documentation for Model Governance

MKM Research Labs

Version 1.0 — 04-April-2026

David K Kelly

Legal Notice

PROPRIETARY AND CONFIDENTIAL

1 Test Results — Flood Polynomial Model (MKM-FPO-001)

Tests run on **04 April 2026 at 14:48:33**.

Table 1: Test Results Summary — Flood Polynomial Model (MKM-FPO-001)

Total Tests	33
Passed	33
Failed	0
Skipped	0
Duration	0.00s

Table 2: Detailed Test Results — Flood Polynomial Model

Test Description	Acceptance Criteria	Result	Time
Coefficients used by model match config/models.py.	`hasattr(flood_poly_coeffs, 'a'); hasattr(flood_poly_coeffs, 'f')` (+1 more)	Pass	0.000s
Fit metadata from config	`flood_poly_fit["model_id"] == "flood-poly-v1"; flood_poly_fit["r_squared"] > 0.9`	Pass	0.000s
Log eps from config	`flood_poly_log_eps == 1e-8`	Pass	0.000s
Storm hours from config	`flood_poly_storm_hours == 168`	Pass	0.000s
Coefficients count	`card["num_parameters"] == 6`	Pass	0.000s
Fit quality has metrics	`"r_squared" in fq; "mae" in fq` (+1 more)	Pass	0.000s
Has required fields	`"model_id" in card; "version" in card` (+3 more)	Pass	0.000s
Model id matches config	`card["model_id"] == flood_poly_fit["model_id"]`	Pass	0.000s
Returns dict	`isinstance(card, dict)`	Pass	0.000s
Water far above severe at late hour should be near 1.0.	`p > 0.9`	Pass	0.000s
Water well above severe threshold should give high P(flood).	`p > 0.8`	Pass	0.000s
hour=167 (last storm hour) should not raise.	`0.0 <= p <= 1.0`	Pass	0.000s
hour=0 should not raise.	`0.0 <= p <= 1.0`	Pass	0.000s
P(flood) at hour 0 should be lower than mid-storm for moderate water.	`p_early < p_mid`	Pass	0.000s
In range	`0.0 <= result <= 1.0; 0.0 <= result <= 1.0`	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Water well below severe threshold should give low P(flood).	`p < 0.1`	Pass	0.000s
Higher water level should give higher P(flood) at same hour.	`p_low < p_mid < p_high`	Pass	0.000s
Negative severe returns zero	`p_flood_poly(water_level=5.0, hour=48, severe_level=-1.0)...; p_flood_simple(5.0, -1.0) == 0.0`	Pass	0.000s
Returns float	`isinstance(result, float); isinstance(result, float)`	Pass	0.000s
Near-zero water level should give near-zero P(flood).	`p < 0.01`	Pass	0.000s
Zero severe returns zero	`p_flood_poly(water_level=5.0, hour=48, severe_level=0.0) ...; p_flood_simple(5.0, 0.0) == 0.0`	Pass	0.000s
All in range	`all(0.0 <= p <= 1.0 for p in result)`	Pass	0.000s
Correct length	`len(result) == 5`	Pass	0.000s
Empty list	`p_flood_series([], severe_level=6.58) == []`	Pass	0.000s
Returns list	`isinstance(result, list)`	Pass	0.000s
Above severe capped at one	`p_flood_simple(10.0, 6.58) == 1.0`	Pass	0.000s
At severe equals one	`p_flood_simple(6.58, 6.58) == 1.0`	Pass	0.000s
In range	`0.0 <= result <= 1.0; 0.0 <= result <= 1.0`	Pass	0.000s
Negative severe returns zero	`p_flood_poly(water_level=5.0, hour=48, severe_level=-1.0)...; p_flood_simple(5.0, -1.0) == 0.0`	Pass	0.000s
P = min(1, w/s).	`abs(p_flood_simple(3.29, 6.58) - 0.5) < 0.01`	Pass	0.000s
Returns float	`isinstance(result, float); isinstance(result, float)`	Pass	0.000s
Zero severe returns zero	`p_flood_poly(water_level=5.0, hour=48, severe_level=0.0) ...; p_flood_simple(5.0, 0.0) == 0.0`	Pass	0.000s
Zero water	`p_flood_simple(0.0, 6.58) == 0.0`	Pass	0.000s

1.1 Test Document History

Date	Version	Description	Author
04-Apr-2026	1.0	Initial test suite (33 tests)	D. Kelly